DOCUMENTATION TECHNIQUE

SUDRIBET

Application mobile de paris sportifs universitaires sur plateforme Android

● Version 1.0 · Android 7.0+ (API 24) · Kotlin 2.2.0

AUTEURS

— Julian Expert
— Malo Greffier
— Sacha Lathuillière

Année académique 2025–2026
ESME Sudria

Table des matières

1. Présentation du projet 3
   - Contexte et objectifs
   - Périmètre fonctionnel
2. Architecture technique 4
   - Stack technologique
   - Organisation des couches
   - Flux de données
3. Fonctionnalités détaillées 6
   - Authentification · Accueil · Paris sportifs · Historique · Profil · SudriBot IA · Classement
4. Modèles de données 9
5. Modules métier 10
   - BetResolver · DailyManager · BadgeSystem
6. Dépendances & librairies 12
7. Guide d'installation 13
8. Limitations & perspectives 14

1 Présentation du projet

Contexte et objectifs

SudriBet est une application mobile Android développée dans le cadre du cours de développement mobile de l'ESME Sudria (2025–2026). Elle simule une plateforme de paris sportifs universitaires permettant aux étudiants de parier sur des matchs opposant les grandes écoles d'ingénieurs françaises (ESME, EPITA, IPSA, Centrale, etc.) à l'aide d'une monnaie virtuelle appelée **SudriCoin (SC)**.

L'application n'implique aucun argent réel. Son objectif pédagogique est de mettre en œuvre une architecture Android moderne combinant vues XML, Jetpack Compose, une IA générative (Google Gemini 2.0) et Firebase Analytics.

> **Note :** Toute transaction dans SudriBet est fictive. Le solde en SudriCoins (SC) ne possède aucune valeur monétaire réelle.

Périmètre fonctionnel

🔐

Authentification

Connexion par email, session persistante via SharedPreferences

🏠

Accueil

Dashboard avec solde, missions du jour, widget IA et classement

⚽

Paris sportifs

Matchs en temps réel, paris simples et combinés

📋

Historique

Liste des paris passés avec statuts et gains

👤

Profil

Photo de profil, badges, statistiques, niveau XP

🤖

SudriBot IA

Assistant propulsé par Google Gemini 2.0 Flash

2 Architecture technique

Stack technologique

COMPOSANT	TECHNOLOGIE	VERSION	RÔLE
Langage	**Kotlin**	2.2.0	Développement de toute la logique applicative
UI principale	**Android Views XML**	—	Mise en page des écrans principaux
UI moderne	**Jetpack Compose**	BOM 2024.12	Liste de matchs avec filtres par sport
IA générative	**Google Gemini 2.0 Flash**	SDK 0.7.0	Chatbot SudriBot, analyse de matchs
Analytics	**Firebase Analytics**	BOM 34.11.0	Suivi des événements utilisateur
Sérialisation	**Gson**	2.10.1	Persistance locale JSON dans SharedPreferences
Réseau	**Retrofit + OkHttp**	2.9.0 / 4.10.0	Client HTTP extensible
Animations	**Lottie**	6.4.0	Animations vectorielles JSON
Build system	**Gradle Kotlin DSL**	9.0.0	Compilation et gestion des dépendances
Min SDK	**Android 7.0**	API 24	Compatibilité minimale
Target SDK	**Android 15**	API 35	Version cible — testé sur Xiaomi (Android 15)

Organisation des couches

COUCHE PRÉSENTATION (ACTIVITIES + ADAPTERS)

`LoginActivity` `HomeActivity` `MainActivity` `HistoryActivity` `ProfilActivity` `EditProfilActivity` `ChatActivity` `LeaderboardActivity` `MatchDetailActivity` `BetAdapter` `MatchAdapter` `LeaderboardAdapter`

↕

COUCHE LOGIQUE MÉTIER (MANAGERS + VIEWMODEL)

`MatchViewModel` `BetResolver` `DailyManager` `BadgeSystem` `ActivityTransitions` `LocalNotificationHelper` `EmailService`

↕

COUCHE DONNÉES & SERVICES EXTERNES

`SharedPreferences (JSON)` `Firebase Analytics` `Gemini API` `filesDir/avatar.jpg`

Flux de données

SudriBet adopte un stockage **100 % local** via `SharedPreferences`. Les paris sont sérialisés en JSON (Gson) sous la clé `my_bets`. Le solde est stocké en `Float` sous la clé `balance`. La photo de profil est copiée dans le répertoire interne privé (`filesDir/avatar.jpg`) et son chemin absolu est persisté.

> **Navigation :** L'objet singleton `ActivityTransitions` gère toutes les transitions. Il sélectionne automatiquement `overrideActivityTransition` (API 34+) ou `overridePendingTransition` selon la version Android.

3 Fonctionnalités détaillées

🔐 Authentification — `LoginActivity`

Saisie d'un email et d'un mot de passe (validation locale uniquement). L'email est persisté dans SharedPreferences. Au lancement suivant, si la clé `email` existe, l'utilisateur est redirigé directement vers l'accueil (auto-login). La déconnexion supprime la clé et efface la pile de navigation.

🏠 Accueil — `HomeActivity`

Dashboard affichant le solde, une salutation personnalisée, des raccourcis rapides, le widget SudriBot IA, les trois missions du jour, le classement et une actualité sportive. Recharge automatique de 150 SC si solde < 50 SC. Bonus aléatoire de 25 SC (20 % de chance) via le bouton étoile, une fois par jour.

⚽ Paris sportifs — `MainActivity` + `MatchViewModel`

Liste de 12 matchs universitaires gérée par un ViewModel avec Kotlin Flow. Les matchs live ont leurs scores mis à jour toutes les 3 secondes. Filtre par sport et barre de recherche disponibles. La liste est rendue en Jetpack Compose via un ComposeView imbriqué. Un pari combiné multiplie les cotes. La mise est déduite immédiatement lors de la validation.

📋 Historique des paris — `HistoryActivity`

RecyclerView affichant tous les paris (rechargé à chaque `onResume`). Chaque item affiche date, statut coloré, description, mise, cote et gains potentiels. Un état vide avec bouton d'action est affiché si aucun pari n'a encore été placé.

👤 Profil — `ProfilActivity` + `EditProfilActivity`

Affichage du pseudo, solde, statistiques (victoires, défaites, taux de réussite), niveau XP et badges débloqués. La photo de profil est sélectionnable depuis la galerie via `ActivityResultContracts.GetContent`, copiée en stockage interne privé et affichée partout dans l'app (accueil, profil, édition).

🤖 SudriBot IA — `ChatActivity`

Interface de chat avec Google Gemini 2.0 Flash, configuré avec un system prompt spécialisé en paris universitaires. SudriBot peut proposer des paris sous forme d'un bloc JSON structuré `[[BET_JSON:{…}]]` que l'app parse pour afficher un bouton de pari direct. Clé API injectée au build via `BuildConfig.GEMINI_API_KEY`.

🏆 Classement — `LeaderboardActivity`

Tableau de 10 joueurs dont l'utilisateur connecté est inséré dynamiquement selon son solde réel. Trié par solde décroissant. Podium avec médailles emoji pour les 3 premières places. L'entrée de l'utilisateur est mise en surbrillance.

4 Modèles de données

Bet — Pari

```
data class Bet(
    val id              : String,  // UUID unique
    val description     : String,  // ex : "ESME vs EPITA (1), CentraleSupélec vs ESME (X)"
    val totalCote       : Double,  // cote combinée (produit des cotes individuelles)
    val mise            : Double,  // montant misé en SudriCoins
    val gainsPotentiels : Double,  // totalCote × mise
    val date            : String,  // format "dd/MM HH:mm"
    val status          : String   // "En cours" | "Gagné" | "Perdu"
)
```

Match — Rencontre sportive

```
data class Match(
    val id        : String,          // identifiant numérique
    val equipeA   : String,          // équipe domicile
    val equipeB   : String,          // équipe extérieure
    val coteA     : Double,          // cote victoire A
    val coteB     : Double,          // cote victoire B
    val heure     : String,          // "HH:MM"
    val categorie : String,          // "Football" | "Rugby" | "Basket" | "Handball" | "Volley"
    val isLive    : Boolean = false, // score mis à jour toutes les 3 secondes
    val scoreA    : Int     = 0,
    val scoreB    : Int     = 0,
    val coteNul   : Double? = null   // nul disponible uniquement au football
)
```

Persistance — Clés SharedPreferences

CLÉ	TYPE	DESCRIPTION
`email`	String	Email connecté (session active)
`username`	String	Pseudo affiché
`balance`	Float	Solde en SudriCoins
`my_bets`	String (JSON)	Liste sérialisée `List<Bet>`
`avatar_path`	String	Chemin absolu vers `filesDir/avatar.jpg`
`dark_mode`	Boolean	Mode sombre activé
`last_recharge_date`	String yyyy-MM-dd	Date dernière recharge automatique
`last_bonus_date`	String yyyy-MM-dd	Date dernier bonus étoile
`bets_today_YYYY-MM-DD`	Int	Nombre de paris placés ce jour
`mission_claimed_DATE_N`	Boolean	Récompense mission N déjà réclamée

5 Modules métier

BetResolver — Résolution des paris

Objet singleton appelé à chaque démarrage et reprise de `HomeActivity`. Il itère sur tous les paris « En cours » et leur affecte un résultat selon une probabilité inversement proportionnelle à la cote.

```
// Probabilité inversement proportionnelle à la cote
val winProbability = 1.0 / bet.totalCote
val won = Random.nextDouble() < winProbability
// Cote 2.0 → 50%  |  Cote 5.0 → 20%  |  Cote 1.5 → 67%
```

En cas de victoire, les gains potentiels sont crédités sur le solde et une notification locale est envoyée via `LocalNotificationHelper`.

DailyManager — Gestion quotidienne

MÉTHODE	DESCRIPTION	DÉCLENCHEMENT
`checkDailyRecharge()`	Crédite 150 SC si solde < 50 SC, une fois/jour	`HomeActivity.onCreate()`
`checkDailyBonus()`	Attribue 25 SC avec 20 % de probabilité, une fois/jour	Clic bouton ⭐
`getMissions()`	Retourne les 3 missions du jour avec progression	`HomeActivity.onCreate()`
`claimMissionReward()`	Crédite la récompense d'une mission (idempotent)	Auto à l'affichage si complétée
`incrementBetsToday()`	Incrémente le compteur de paris du jour	Après validation d'un pari

MISSIONS QUOTIDIENNES

#	MISSION	OBJECTIF	RÉCOMPENSE
1	Place 3 paris aujourd'hui	3 paris	+20 SC
2	Gagne un pari	1 victoire	+30 SC
3	Parie sur un match ESME	1 pari avec « (»	+15 SC

BadgeSystem — Système de récompenses

`BadgeSystem.getBadges()` calcule dynamiquement les 10 badges à chaque affichage du profil.

EMOJI	NOM	CONDITION
🎯	Premier pari	total ≥ 1
🏆	Première victoire	victoires ≥ 1
🔥	Série x3	3 victoires consécutives
💎	Combiné	Un pari combiné placé
🎰	Accro	total ≥ 10 paris
⭐	Lucky	victoires ≥ 5
🚀	Expert	victoires ≥ 10
💰	Riche	solde ≥ 500 SC
🎳	Combo Master	Pari de 3+ matchs
👑	Légende	victoires ≥ 20

6 Dépendances & librairies

LIBRAIRIE	VERSION	USAGE	CATÉGORIE
androidx.core:core-ktx	1.15.0	Extensions Kotlin Android	AndroidX
androidx.appcompat	1.7.0	Compatibilité thèmes Material	AndroidX
material	1.12.0	Composants Material Design 3	UI
activity-compose	1.9.3	Intégration Compose dans Activity	Compose
compose-bom	2024.12.01	Bill of Materials Jetpack Compose	Compose
material3 (Compose)	(BOM)	Composants Material 3 Compose	Compose
firebase-bom	34.11.0	Bill of Materials Firebase	Firebase
firebase-analytics	(BOM)	Tracking des événements	Firebase
generativeai (Gemini)	0.7.0	SDK Google Gemini — SudriBot	IA
lottie	6.4.0	Animations vectorielles JSON	Animations
retrofit2	2.9.0	Client HTTP type-safe	Réseau
okhttp3:logging-interceptor	4.10.0	Logging des requêtes HTTP	Réseau
gson	2.10.1	Sérialisation / désérialisation JSON	Données
lifecycle-runtime-ktx	2.8.7	ViewModel + Coroutines	AndroidX
constraintlayout	2.2.0	Mise en page contrainte	UI

7 Guide d'installation

Prérequis

- Android Studio Ladybug (2024.2) ou supérieur
- JDK 17 (inclus dans Android Studio)
- Android SDK 35 installé
- Appareil physique ou émulateur Android 7.0+ (API 24+)
- Compte Google AI Studio pour la clé API Gemini (optionnel)
- Projet Firebase créé avec `google-services.json`

Étapes de configuration

1 — OUVRIR LE PROJET

```
# Android Studio → File → Open → dossier application_mobile/
# Attendre la synchronisation Gradle automatique
```

2 — CONFIGURER LA CLÉ API GEMINI

```
# Fichier local.properties (racine du projet) :
GEMINI_API_KEY=votre_cle_api_google_gemini
```

Important : `local.properties` est exclu du contrôle de version. Ne jamais committer ce fichier.

3 — AJOUTER LE FICHIER FIREBASE

```
# Télécharger google-services.json depuis console.firebase.google.com
# Le placer dans : app/google-services.json
```

4 — COMPILER ET DÉPLOYER

```
# Android Studio :
# Build → Make Project    (Ctrl+F9)
# Run → Run 'app'         (Shift+F10)

# Ligne de commande (depuis application_mobile/) :
JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home" \
  ./app/gradlew installDebug
```

Compte de test : saisir n'importe quel email et mot de passe lors de la première connexion. L'authentification est locale. Solde initial : **150 SC**.

8 Limitations connues & perspectives

Limitations actuelles

COMPOSANT	LIMITATION	IMPACT
Authentification	Aucune vérification serveur, mot de passe non haché	Moyen
Données matchs	Liste statique de 12 matchs, pas d'API temps réel	Moyen
Persistance	SharedPreferences uniquement — pas de base de données	Faible
Multi-utilisateur	Un seul profil par installation	Faible
Gemini API	Quota gratuit limité (~15 req/min sur le plan Free)	Moyen
Résolution des paris	Algorithme purement aléatoire, sans données sportives	Faible
Synchronisation	Aucun cloud — données perdues si désinstallation	Moyen

Pistes d'amélioration

☁️

Backend Cloud

Firebase Firestore pour synchronisation multi-appareils et classement global en temps réel

🔑

Authentification sécurisée

Migration vers Firebase Authentication (OAuth2 / Google Sign-In)

📡

Données sportives réelles

Intégration d'une API sportive (TheSportsDB, API-Football) pour des cotes et scores live

🎨

Migration Compose

Réécriture complète de l'UI en Jetpack Compose pour cohérence et animations enrichies

🔔

Notifications push

Firebase Cloud Messaging pour notifier les résultats de paris en temps réel

🌍

Internationalisation

Support multilingue via les ressources `values-en`, `values-es`

Testé sur : Xiaomi 24094RAD4G — Android 15.0 (API 35, arm64-v8a) — Build successful, 0 erreur.

Julian Expert · Malo Greffier · Sacha Lathuillière